

fpos Requirements

- [Introduction](#)
- [Discussion](#)
- [Resolved Issues](#)
- [Proposed Resolution](#)
- [Bibliography](#)
- [Acknowledgements](#)

Introduction

This is a plain wording fixing proposal for the table-based requirement set of class template `fpos`. A paper is provided instead of drafting wording in the referenced issues, to have more room available for presentation and explanation.

The wording changes suggested by this paper *should not* have effects on any existing implementations, because it mainly intended to clarify several underspecified parts of the current specification. Nonetheless during the analysis of the paper there occurred some curiosities in the specification that caused to ask some questions to the committee which *might* lead to decisions to change existing implementations, but the author has tried to be as conservative as possible in regard to such suggestions.

Discussion

The `fpos` type family is one of the rarely and sparsely explicitly mentioned templates of the Standard Library specified in 30.5.4.2 [fpos.operations] including Table 112 — "Position type requirements" (all references within this proposal to the C++ working draft are based on ¹). Specializations of that template are used in the IO Library to denote file positions and to allow some arithmetic operations and to store/retrieve a rather opaque "state"; furthermore it connects `std::char_traits` with the IO streams due to its `pos_type` member which is an `fpos` instantiation whose state type is equal to `char_traits<CharT>::state_type`. The latter property is not a strict requirement, since the specification leaves it implementation-defined what happens when `traits::pos_type` differs from `fpos<char_traits<CharT>::state_type>`. The current wording also says (30.9.2 [filebuf] p4) that the behaviour is undefined, if `traits::pos_type` of `basic_filebuf` is different from `fpos<traits::state_type>`. For the pre-defined `char_traits`, `traits::state_type` is equal to `mbstate_t`, just another sparsely-specified type inherited from the `<wchar>` header. The C specification tells us, that `mbstate_t` is a complete, non-array object type that can be default-initialized and value-initialized (freely translated into C++ language terms) and "can hold the conversion state information necessary to convert between sequences of multibyte characters and wide characters". In C++ (see 24.2.2 [char.traits.typedefs] p4) the requirements imposed on `char_traits<>::state_type` are `CopyAssignable`, `CopyConstructible`, and `DefaultConstructible`. Associated with both `char_traits` and `fpos` are also the types `streamoff` and `streamsize` which are both signed basic integral types (30.5.2 [stream.types] p1+2). An alias for `std::fpos<std::mbstate_t>` is `std::streampos`.

1. Specification Facts

Summarizing the essence of 30.5.4.1 [fpos.members] and 30.5.4.2 [fpos.operations], we can conclude the following:

- `fpos<stateT>` holds a single object of type `stateT`, that can be read from and written to via a state attribute.
- `fpos<stateT>` objects are `EqualityComparable` (This is not explicitly said but it can be indirectly deduced from the definition of `EqualityComparable` in Table 20 and the imposed requirements of `fpos` in regard to `==` in Table 112).
- `fpos<stateT>` objects can be direct-initialized or copy-initialized from `int` values.
- `fpos<stateT>` objects `p` can be direct-initialized from `streamoff` objects `o` and `streamoff` objects can be direct-initialized from `fpos<stateT>` objects, such that the conversion sequence $p_1 \rightarrow o \rightarrow p_2$ ensures that $p_1 == p_2$.
- `streamoff` values can be added to and subtracted from `fpos` and the difference of two `fpos` values returns a `streamoff`.

2. Specification Problems

The following bullets list some concrete unclear points. To simplify the descriptions below, `P` denotes a class type that is an instance of the `std::fpos` template and `O` denotes the type `std::streamoff`.

1. What is the semantics of initializing an `fpos` from an `int` value? (see [LWG 2832](#)). Note that contrary to the construction from `streamoff` there exists no similar conversion sequence cycle guarantee for `int` values. In theory, an (insane) implementation could provide different constructors of `fpos` for `int` and for `streamoff`, both having different semantics. Or, the constructor taking an `int` could be non-explicit and the constructor taking an `streamoff` could be explicit. The author is convinced that this interpretation is not intended and recommends to specify the semantics of `fpos` initialization from `int` to be equivalent to the semantics of `fpos` initialization from `streamoff`, where the `streamoff` value has been initialized from `int` before (Note that due to the type nature of `streamoff` this conversion is statically possible, and it should always be dynamically be well-defined if the conversion of `int` to `streamoff` is well-defined). The indirect effect of doing that is that the specification would now relax the direct-initialization requirement from `O` to `P` to an implicit conversion from `O` to `P`, which is in sync with what existing implementations do anyway.
2. Related to the previous item, some entries in Table 112 are very likely debris of the last cleanup paper in this area, specifically by [N2884](#). Before that paper, type `streamoff` was an implementation-defined type, not necessarily a built-in type. With the resolution of that paper `streamoff` became a guaranteed alias for a signed integral type. Since all integer types can be implicitly converted into each other, there is not much reason to provide the *additional* guarantee that `fpos` instances can be initialized by `int` values, because relying on the initialization by `streamoff` should be sufficient (This assumes the absence of implementations that used SFINAE-constrained constructors to accept *exactly* type `streamoff` for one constructor and *exactly* type `int` for another constructor). For similar reasons is the last row in Table 112 obsolete, which specifies the validity to convert between `streamsize` and `streamoff`, because `streamsize` is (and was) also a synonym for a signed integral type. This proposal therefore recommends to remove this last row as well as the explicit initialization specification of `fpos` by `int` values.
3. What requirements are imposed on the template type parameter `stateT`? (see [LWG 2808](#)). The current specification allows to conclude that `stateT` is required to be `DefaultConstructible`, because there exists no portable way to initialize `fpos` with `stateT`, assuming that an implementation is not required to in-place construct `stateT`. The semantics of the state attribute functions allows the assumption that `stateT` needs to be also `CopyConstructible`, `CopyAssignable`, and `Destructible`. This overlaps nicely

with the requirements that come from 24.2.2 [char.traits.typedefs] p4, but adds a missing Destructible requirement.

A related open question is that the current specification doesn't tell whether the specified `fpos<stateT>` constructors will default-initialize or value-initialize the exposition-only `stateT` member, because this has impact on conforming programs that attempt to read the state before calling the setter. After inspection of existing implementations it turns out that all do the same thing, they *value-initialize* the state member. The author therefore recommends to specify this existing practice. If the committee would consider this as an unwanted strong specification, the wording should at least emphasize that the `stateT` member may be default-initialized after construction and may thus have an indeterminate value, which implies that an initial call of the `stateT state() const` signature would be undefined behaviour.

4. Are `fpos` instances `DefaultConstructible`, `CopyConstructible`, `CopyAssignable`, and `Destructible`? The requirements for `CopyConstructible` and `Destructible` follow indirectly from the usage of `fpos` in return types and function arguments from a pre-C++11 era, where every move was a copy. There is currently clearly no `DefaultConstructible` requirement, but existing implementations such as from Visual Studio 2017/Dinkumware, clang/libc++, and gcc/libstd++ provide default-constructible `fpos` types. In all cases the default-constructed `fpos` is equivalent to an `fpos` initialized by `streamoff(0)`. Therefore this proposal suggests to require that `fpos` instances are `CopyAssignable` and `DefaultConstructible`.
5. Existing implementations of `fpos` provide implicit conversion to `streamoff`, the specification requires only explicit conversions. The current implementation state can be explained by the pre-C++11 existence of that type, where no explicit conversion functions were available. Nonetheless the author believes that the currently stricter specification is reasonable for now and doesn't recommend any change. The author is open to be requested to weaken the specification, therefore points this characteristics out here, asking for feedback from the committee.
6. Are there any trivial type guarantees of `fpos`, conditionally depending on triviality characteristics of `stateT`? Albeit current implementations provide a default-constructor for `fpos`, this one is never *trivial*. So regardless whether our recommendation is accepted to require the existence of an `fpos` default constructor, we can't give trivial type guarantees. But all implementations available to the author meet the following characteristics:

```
static_assert(std::is_trivially_copyable_v<std::streampos>);
static_assert(!std::is_trivial_v<std::streampos>);
```

and thus are *trivially copyable* if `stateT` is also trivially copyable. The author of this proposal suggest to provide individual trivial special member guarantees depending on corresponding `is_trivially_xxx<stateT>` properties, similar to the special member specifications of `std::istream_iterator`.

7. The mixed additions of `fpos` and `streamoff` are not symmetric (`fpos` on the right side is not specified), is this intended?

Let `p` denote a (possibly `const`) value of `P` and `o` denote a (possibly `const`) value of `O`, the existing requirements support the following expression

```
p + o;
```

but *not* the other way around:

```
o + p;
```

The observation that `o + p` is undefined by the letters of the Standard is unexpected for users and according to the author's opinion user code uses either form obviously unaware of that fact. The interesting point is that all implementations seem to behave the same way as follows:

```

P p(0);
O o(0);

using add1 = decltype(p + o);
using add2 = decltype(o + p);

static_assert(std::is_same_v<add1, P>);
static_assert(std::is_same_v<add2, O>);

```

This program is well-formed for all tested implementations, which means that `p + o` and `o + p` do have different result types: The first one has type `P`, the second one has type `O`. The technical reason for this is that existing implementations only define a member signature `fpos operator+(streamoff offs) const` but also always a conversion function `operator streamoff() const`, the net result being that the expression `o + p` is well-formed, because it finds the conversion function and after the conversion of `p` to `streamoff` it just adds two `streamoff` values.

The author of this proposal believes that the binary plus operation of `P` and `O` should be well-defined to reduce user surprises. Unfortunately existing implementations behave non-symmetric in regard to the semantics and result type. It is unclear how to proceed here:

1. Keep the current state and leave `o + p` undefined. User-code cannot call that combination of this binary operator at all in conforming code.
2. Change the specification to make `o + p` well-defined, allow one of `P` or `O` to be valid. User-code could thus either use `auto` for the result or rely on an implicit conversion to `P` in conforming code.
3. Change the specification to make `o + p` well-defined, and say the the return type is *convertible* to `P`. User-code could thus either use `auto` for the result or rely on an implicit conversion to `P` in conforming code.
4. Change the specification to make `o + p` well-defined, but specify what existing implementations do. User-code could thus either use `O` for the result or rely on implicit conversion to `P` in conforming code.
5. Change the specification to make `o + p` well-defined and require that all existing implementations change so that the return type is `P` instead of `O`. User-code could thus either use `P` for the result or rely on direct initialization to `O` in conforming code.

Some short observations to some of the suggested variants: Variant (2) would be a very unusual specification, because we typically don't say that the result is one of several choices. In diffuse cases we typically make an implicit conversion to some other type part of the spec (See for example the result of `==` or `!=` in most requirement sets). Interestingly, all variants have in common that the result is convertible to `P`, which is appealing because it emphasizes the "right" type. Variant (3) uses this commonality as the specification directly.

The author would like to gather feedback from the committee in regard to the preferred outcome, but suggests to use the non-breaking variant (3), which has the following advantages: First, its specification emphasizes the intuitively "correct" result type. Second, if the committee is in fond of that idea, existing implementations could decide in the future to *add* the missing `P` `operator+(O, P)`. Then, in a later future, if a new survey of existing practice would be performed again and when now all existing implementations have added this member, we could now strengthen the specification to impose that the return value of `o + p` is `P`.

The author would also express his recommendation *not* to standardize the behaviour of existing implementations (variant (4)). This seems an unusual recommendation at first, but in this particular case, the author suspects that the status quo of implementations doesn't exist because of an intentional design,

but due to an *unintended* artifact of strictly implementing what Table 112 describes. The author would like to get feedback from existing implementations whether this assumption is correct or wrong.

8. According to Table 112 it looks as if the return types of assignment operator and compound assignment operator of `fpos` have type `fpos` and not `fpos&` (similar for `streamoff`). The presumable intend is to specify the return types of the pure binary arithmetic expressions and not of the denoted assignments (except for the compound assignments). This paper proposes to isolate the different expressions and fix the wrong compound assignment return types.
9. Some rows in Table 112 specify a return type `fpos` for expressions such as `P(o)`. This is invalid, because `fpos` is a class template, but the return type for an expression `P(o)` needs to be `P`, which is the same *specialization* of `fpos` that has been used to form the actual expression. The recommended resolution is to replace such uses of `fpos` in Table 112 by `P` instead.

Resolved Issues

If the proposed resolution will be accepted, the following library issues will be resolved:

Number	Description	C++17 NB comment
2808	Requirements for <code>fpos</code> and <code>stateT</code>	GB 60
2832	§[fpos.operations] strange requirement for <code>P(i)</code>	—

Proposed resolution

At some places below, additional markup of the form

[Drafting notes: whatever — end drafting notes]

is provided, which is *not* part of the normative wording, but is solely shown to provide additional information to the reader about the rationale of the concrete wording.

The proposed wording changes refer in all cases to [N4659](#).

1. Change 30.5.4.2 [fpos.operations] as indicated:

~~-1- Operations specified in Table 112 are permitted.~~ An `fpos` type specifies file position information. It holds a state object whose type is equal to the template parameter `stateT`. Type `stateT` shall meet the `DefaultConstructible` (Table 22), `CopyConstructible` (Table 24), `CopyAssignable` (Table 26), and `Destructible` (Table 27) requirements. If `is_trivially_copy_constructible_v<stateT>` is true, then `fpos<stateT>` has a trivial copy constructor. If `is_trivially_copy_assignable<stateT>` is true, then `fpos<stateT>` has a trivial copy assignment operator. If `is_trivially_destructible_v<stateT>` is true, then `fpos<stateT>` has a trivial destructor. All specializations of `fpos` satisfy the `DefaultConstructible`, `CopyConstructible`, `CopyAssignable`, `Destructible`, and `EqualityComparable` (Table 20) requirements. In addition, the expressions shown in Table 112 are valid and have the indicated semantics. In that table,

(1.1) — `P` refers to an instance of `fpos`,

(1.2) — `p` and `q` refer to values of type `P` or `const P`,

(1.?) — `p1` and `q1` refer to modifiable lvalues of type `P`,

(1.3) — `o` refers to type `streamoff`,

(1.4) — `o` refers to a value of type `streamoff` or `const streamoff`, and

(1.?) — `o1` refers to a modifiable lvalue of type `streamoff`.

(1.5) — `sz` refers to a value of type `streamsize` and

(1.6) — `i` refers to a value of type `int`.

[Drafting notes: It is recommend to strike the non-normative note in p2 completely. This seems to be very out-dated wording and what is says is already said in 20.4.1.3 [structure.requirements], in particular in p3+p4 — end drafting notes]

~~2- [Note: Every implementation is required to supply overloaded operators on `fpos` objects to satisfy the requirements of 30.5.4.2 [fpos.operations]. It is unspecified whether these operators are members of `fpos`, global operators, or provided in some other way. — end note]~~

-3- Stream operations that return a value of type `traits::pos_type` return `P(0(-1))` as an invalid value to signal an error. If this value is used as an argument to any `istream`, `ostream`, or `streambuf` member that accepts a value of type `traits::pos_type` then the behavior of that function is undefined.

2. Change Table 112 — "Position type requirements", as indicated:

[Drafting notes: We can strike the wording about the destructor and `==`, because we have now the `EqualityComparable` and `Destructible` requirements specified in 30.5.4.2 [fpos.operations] — end drafting notes]

Table 112 — Position type requirements

Expression	Return type	Operational semantics	Assertion/note pre-/post-condition
<code>P(i)</code>	—	—	<code>p == P(i)</code> note: a destructor is assumed.
<code>P p(i);</code> <code>P p = i;</code>	—	—	Postconditions: <code>p == P(i)</code>.
<code>P(o)</code>	<code>fpos</code> <code>P</code>	converts from offset	<u>Effects: Value-initializes the state object.</u>
<code>P p(o);</code> <code>P p = o;</code>			<u>Effects: Value-initializes the state object.</u> <u>Postconditions: <code>p == P(o)</code>.</u>
<code>P(.)</code>	<code>P</code>	<code>P(0)</code>	
<code>P p;</code>		<code>P p(0);</code>	
<code>O(p)</code>	<code>streamoff</code>	converts to offset	<code>P(O(p)) == p</code>
<code>p == q</code>	convertible to	—	<code>==</code> is an equivalence relation

	bool		
p != q	convertible to bool	!(p == q)	
q = p + o p += o	fpos P	+ offset	<u>Remarks: With ql = p + o;, then:</u> <u>ql - o == p</u>
pl += o	P&	+= offset	<u>Remarks: With ql = pl; before the</u> <u>+=, then:</u> <u>pl - o == ql</u>
q = p - o p -= o	fpos P	- offset	<u>Remarks: With ql = p - o;, then:</u> <u>ql + o == p</u>
pl -= o	P&	-= offset	<u>Remarks: With ql = pl; before the</u> <u>-=, then:</u> <u>pl + o == ql</u>
o + p	convertible to P		<u>P(o + p) == p + o</u>
o = p - q	streamoff	distance	<u>p == q + (p - q).</u> <u>Remarks: With ol = p - q;, then:</u> <u>q + ol == p</u>
streamsize(o) o(sz)	streamsize streamoff	converts converts	streamsize(o(sz)) == sz streamsize(o(sz)) == sz

Bibliography

[N4659 Richard Smith: "Working Draft, Standard for Programming Language C++"](#)

Acknowledgements

Thanks to Jonathan Wakely and Billy Robert O'Neal III for a review of this proposal and for encouragement to make a little more changes as I originally believed I should make.